F# Units of Measure: Implementation and Type-Theoretic Soundness

Francesco Borri, matricola 639079
f.borri2@studenti.unipi.it

Contents

1 Introduction

In F#, units of measure, or *measures* for short, are a **static type-system extension** that allows numeric values to be annotated with physical or abstract measures (e.g., metres, seconds). These annotations are checked at compile-time and prevent invalid operations, such as adding quantities with incompatible dimension or forgetting necessary conversions.

Conceptually, a unit of measure is not a run-time entity but instead a **compile-time type-level construct**. For example, a value of type `float<m>`

(which should be read as the type `float` annotated with the measure `m`) is distinct from `float<s>`, even though both are represented as plain floating-point numbers at run-time. The compiler tracks measures **symbolically** and enforces algebraic rules (multiplication, division, exponentiation) over them.

1.1 Motivations, historical notes and related work

The primary motivation behind the introduction of this feature, released in 2009 in F# 2.0 [4], was to address a well-known class of bugs (*dimensional errors*), which are hard to debug due to their subtle nature. In fact, a measure-mismatch error is related to the way we interpret the data (as humans) managed by software: regardless of whether something represents cubic metres or newtons, the underlying representation will still be a floating-point number. This is the main reason why, sometimes, a dimensionally-incorrect numeric computation may pass the compile-time checks or even execute successfully while producing wrong results.

One of the most famous examples of dimensional error is the failure of NASA's *Mars Climate Orbiter*, a robotic space probe meant to study the atmosphere and climate of Mars. The spacecraft encountered Mars on a trajectory that brought it too close to the planet, causing it to be destroyed in the atmosphere. An investigation attributed the cause of the failure to a misinterpretation of a result, which was calculated in *pound-seconds* but was expected to be in *newton-seconds*, leading to an error of a factor of $\approx 4,44$.

The type-theoretic approach for dealing with dimensional data was mainly inspired by Kennedy's work [3] (as reported in the official publications web-page of F# [1]), which proposed an extension of the typed lambda-calculus with a notion of measure-parametric polymorphism and studied its implications in-depth.

2 Overview

2.1 Declaring and annotating with units of measure

In F#, measures are represented as **type annotations** that extend numeric types with additional information. They are defined using type declaration with special attribute, as shown in Listing 1.

```
[<Measure>] type m  // metre
[<Measure>] type s  // second
[<Measure>] type kg // kilogram
```

Listing 1: Definitions of the measure-annotation for metres, seconds and kilograms.

It is also possible to define derived measures using the same syntax as shown in Listing 2. Derived measures can be created by means of product (which

is represented by `*` or simply by juxtaposition), quotient (`/`) or by raising a measure to a fractional power (using `^`), meaning that the programmer must explicitly write numerator and denominator, as shown in Listing 3.

```
[<Measure>] type N = m * kg / s^2  // Newton
```

Listing 2: Definitions of the measure-annotation for Netwons.

```
[<Measure>] type sqrtm = m ^ (1/2)
```

Listing 3: Definition of the measure-annotation for $\sqrt{m}$.

There is also the possibility to have dimensionless values, which for instance come out from the ratio between two values measuring the same dimension; in F# those values can be annotated using the special measure `1`. Of course, as this type represent dimensionless values, the compiler treats it as the identity of the product, which means that the measures `m * 1`, `1 * m` and `m` are all equivalent.

Units of measure can be attached both to **numeric types** and **numeric literals** using angle brackets, as shown in Listing 4. By default, non-annotated numeric types (and values) are intended as dimensionless; for instance, the type `float` is equivalent to `float<1>`.

```
type metres = float<m>
let a: metres = 1.0<m> * 2.0    // 2.0 is equivalent to 2.0<1>
```

Listing 4: Example of a measure-annotated type and value.

2.2 Expressions and functions

As in physics you can't sum values with different units of measure, the F# compiler doesn't allow you to sum values annotated with different measures, as shown in Listing 5. Instead, it is possible to take the product or the quotient of values with different measures, (exactly as speed is obtained by dividing distance by time) and the compiler automatically infers the type of the result without having to annotate it; Listing 5 shows an example of the compiler's automatic type-inference of this operations.

As in standard F# without measures, in order to calculate the value of an expression the underlying numeric types must be the same: for instance, we can't mix the type `int<m>` with `float<m>`.

The compiler's type-inference can also automatically infer the (annotated) return type of a function given the type of its arguments, as shown in Listing 6.

```
let distance = 10.0<m>
let time = 5.0<s>
let deltaTime = 3.0<s>
let invalid = distance + time           // Error
let speed = distance / time             // float<m/s>
let acceleration = speed / delta_time   // float<m/s^2>
```

Listing 5: Example of compiler's type-checking and type-inference with measure annotations.

```
// Inferred type: float<kg> -> float<m/s^2> -> float<kg m s^-2>
let newton2ndLaw (m: float<kg>) (a: float<m/s^2>) = m * a
```

Listing 6: Implementation of Newton's second law of motion.

2.3 Measure-parametric polymorphism

F# also supports **measure-polymorphic** functions, which can be called providing a value of the correct numeric type annotated with any measure; an example is shown in Listing 7, where the measure-polymorphic type `float` is written as `float<'u>`.

```
let add (x: float<'u>) (y: float<'u>) = x + y
let v1 = 3.0<m/s>
let v2 = 4.0<m/s>
let t = 7.0<s>
add v1 v2   // Correct, returns 7.0<m/s>
add v1 t    // Type error
```

Listing 7: Example of a measure-generic function `add`.

It is also possible to declare **measure-polymorphic data types**, where measure variables behave like type variables; an example is shown in Listing 8.

```
type MeasuredPair<[<Measure>] 'u, [<Measure>] 'v> =
    { First: float<'u>; Second: float<'v> }

type MeasuredTree<[<Measure>] 'u> =
    | Leaf of float<'u>
    | Node of MeasuredTree<'u> * MeasuredTree<'u>
```

Listing 8: Example of a measure-polymorphic record and (discriminated) union.

Numeric literals are treated in a special way, as the only real measure-polymorphic literal is just zero, and it is represented by `0.0<_>` , where the

measure _ is called *anonymous* measure (here the inferred type by the compiler is `float<'u>`, but it works for any numeric type). Instead, for any non-zero literal, say `1`, the expression `1<_>` is statically resolved by the compiler depending on the context[1]. Listing 9 shows these concepts at work; the function `badSumPlusOne` isn't measure-polymorphic as the anonymous measure is resolved to the dimensionless measure `1` at definition-site; instead, the type of the *inlined* function `sumPlusOne` is resolved at call-site, and therefore the anonymous measure can depend on the actual argument, making the function measure-polymorphic.

```
// Inferred type: list<float<'u>> -> float<'u>
let rec sum (xs: float<'u> list) =
    match xs with
    | [] -> 0.0<_>
    | a :: xs' -> a + sum xs'

// Inferred type: list<float> -> float (less general)
let rec badSumPlusOne (xs: float<'u> list) =
    match xs with
    | [] -> 1.0<_>
    | a :: xs' -> a + sum xs'

// Inferred type: list<float<'u>> -> float<'u>
let rec inline sumPlusOne (xs: float<'u> list) =
    match xs with
    | [] -> 1.0<_>
    | a :: xs' -> a + sum xs'
```

Listing 9: Examples of how the measure-polymorphic zero works and how the anonymous measure is resolved by the compiler in different contexts.

2.4 Erasure

After the type-checking phase, the F# compiler *erase* all measure annotations, which has as the positive consequence of the absence of run-time overhead for enforcing dimensional correctness.

3 Type-theoretic formalization

This section aims to briefly discuss the type-theoretical foundations behind the F# implementation of the units of measure type-system extension, which have been mostly influenced by Kennedy's work [3]; we will just overview the main

[1]The motivation behind this design choice lays behind theoretical results better discussed in Section 3.

results without going too further into details, with a particular focus on algorithmic aspects impacting also implementation.

Before that, since Kennedy extended the *Hindley-Milner type-system*, the following section shortly explain what is it and how it enables automatic type-inference.

3.1 Hindley-Milner type-system

The Hindley–Milner type system is a **classical static type system** that supports **parametric polymorphism** and enables complete **type inference** without explicit type annotations. Formally, the syntax is an extension of the simply-typed lambda-calculus extended with a `let` construct, which allows names to have a (parametric) polymorphic type. The HM type-system restricts polymorphic type to be of the form $\forall\alpha_1 \dots \forall\alpha_n.\tau$, where $\alpha_1 \dots \alpha_n$ are type variables and can be instantiated with any concrete type, and τ is not allowed to contain a quantification: this is because the type-inference of a system allowing quantification everywhere in a type expression (which is named System F or polymorphic lambda-calculus) is undecidable.

The set of well-typed terms is determined by a deductive system which consist of set of inference rules, having as conclusion typing-judgments of the form $\Gamma \vdash e : \tau$, meaning that under typing context Γ the term e has type τ.

The two terms shown in Equation 1 explain better how `let`-polymorphism works: e_1 has type $\texttt{num} \times \texttt{bool}$, thanks to the polymorphic type $\forall\alpha.\alpha \to \alpha$ of f, whereas e_2 can not be typed as it would have type $\forall\beta.(\forall\alpha.\alpha \to \beta) \to \beta \times \beta$ and it is not allowed in HM type-system as it contains a quantifier in a non-top-level position.

$$\begin{aligned} e_1 &= \texttt{let } f = \lambda x.x \texttt{ in } (f\ 0, f\ \texttt{false}) \\ e_2 &= \lambda f.(f\ 0, f\ \texttt{false}) \end{aligned} \tag{1}$$

The type-inference algorithm for HM system is named *algorithm W*, and it was built upon a rewriting of the deductive system in a syntax-directed manner, meaning that for every construct of the syntax there is exactly one possible choice of inference rule to apply. Since inference rules may contain type generalization or specialization, the algorithm tries to infer the most general type, collecting type constraints and solving them while building the proof tree. As an example, consider the inference rule for the lambda-abstraction construct, defined in Equation 2. The critical choice is τ_1: since the algorithm always wants to find the most general type, it temporarily assign the type $\forall\alpha.\alpha$ (which is the most general among all types) and possibly collects constraints in the subsequent inference rule applications to refine the type of x.

$$\frac{\Gamma \cup \{x : \tau_1\} \vdash e : \tau_2}{\Gamma \vdash \lambda x.e : \tau_1 \to \tau_2} \tag{2}$$

3.2 Unit of measure extension

3.2.1 Syntax

In order to support measure-annotated type, the syntax is extended with a new syntactic category, *measure expression*: they are formally described in Equation 3, where u denotes a measure variable, U denotes a base measure and $\mathbf{1}$ is the dimensionless measure.

$$\delta ::= u \mid U \mid \mathbf{1} \mid \delta_1 \cdot \delta_2 \mid \delta^{-1} \tag{3}$$

We can also define δ^n as syntactic sugar for n repeated multiplications if n is positive, the inverse of $-n$ repeated multiplication if n is negative or $\mathbf{1}$ if n is 0. This definition of exponentiation has a significant design consequence because exponents are forced to be integers, which is not the case in F#. Later we will see how this influences type-inference and what should be done to allow rational exponents.

The first important difference which distinguish measure expressions from classic types is that the equivalence of two measure expressions is not syntactic, but rather semantic: in fact, they should obey the laws shown in Equation (commutativity); thus, we define the relation $\equiv$ as the congruence satisfying the mentioned laws.

$$\begin{aligned}
\delta_1 \cdot \delta_2 &\equiv \delta_2 \cdot \delta_1 && \text{(commutativity)} \\
(\delta_1 \cdot \delta_2) \cdot \delta_3 &\equiv \delta_1 \cdot (\delta_2 \cdot \delta_3) && \text{(associativity)} \\
\mathbf{1} \cdot \delta &\equiv \delta && \text{(identity)} \\
\delta \cdot \delta^{-1} &\equiv \mathbf{1} && \text{(inverses)}
\end{aligned}$$

As a consequence, the set of all measure expression generated by Equation 1 quotiented by $\equiv$ is a *free Abelian group*; here the term *free* means that the group contains a subset B, namely its *basis*, such that every element in the group can be generated in an unique way by taking a finite set of elements from B and combining them (possibly repeating some element). The existence of a basis implies that every measure expression has a unique normal form, shown in Equation 4, where the basis is the union of the (infinite) set of all measure variables with the (finite) set of base measures.

$$\delta = u_1^{x_1} u_2^{x_2} \dots u_n^{x_n} \cdot U_1^{y_1} U_2^{y_2} \dots U_m^{y_m}, x_i, y_j \in \mathbb{Z} \setminus \{0\} \tag{4}$$

Suppose that the HM type-system under analysis includes a base boolean type `bool` and base numeric type `num`: then we can annotate the latter with any measure expression; therefore, the types of our type-system are the ones generated by Equation 5. As in HM type-system, polymorphic types are indexed by σ and can have quantification only at top-level, but now they can be generic also over measure variables, indexed by u.

$$\begin{gathered}\tau ::= t \mid \texttt{bool} \mid \texttt{num}\langle\delta\rangle \mid \tau_1 \to \tau_2 \\ \sigma ::= \tau \mid \forall t.\sigma \mid \forall u.\sigma\end{gathered} \tag{5}$$

In order to be able to write useful programs, we also assume the existence of some primitive numeric measure-aware operations like the ones shown in Equation 6.

$$\begin{gathered}+, - : \forall u.\texttt{num}\langle u\rangle \to \texttt{num}\langle u\rangle \to \texttt{num}\langle u\rangle \\ * : \forall u_1.\forall u_2.\texttt{num}\langle u_1\rangle \to \texttt{num}\langle u_2\rangle \to \texttt{num}\langle u_1 \cdot u_2\rangle \\ / : \forall u_1.\forall u_2.\texttt{num}\langle u_1\rangle \to \texttt{num}\langle u_2\rangle \to \texttt{num}\langle u_1 \cdot u_2^{-1}\rangle\end{gathered} \tag{6}$$

3.2.2 Typing rules

The basic HM typing rules must be adapted to reflect the equational theory imposed on measure expression: Equation 7 show the necessary typing rules, where r represents numeric constants. We also assume the existence of some numeric value $1_U : \texttt{num}\langle U\rangle$ for every U for building non-dimensionless numeric values by considering the expression $r * 1_U : \texttt{num}\langle U\rangle$.

$$\begin{gathered}\frac{\Gamma \vdash e : \texttt{num}\langle\delta_2\rangle}{\Gamma \vdash e : \texttt{num}\langle\delta_1\rangle}(\delta_1 \equiv \delta_2) \quad \frac{}{\Gamma \vdash r : \texttt{num}\langle\mathbf{1}\rangle}(r \neq 0) \quad \frac{}{\Gamma \vdash 0 : \texttt{num}\langle\delta\rangle} \\ \frac{\Gamma \vdash e : \sigma}{\Gamma \vdash e : \forall u.\sigma}(u \text{ not free in } \Gamma) \quad \frac{\Gamma \vdash e : \forall u.\sigma}{\Gamma \vdash e : \{u \to \delta\}\sigma}\end{gathered} \tag{7}$$

The two rules for typing numeric constants mean that only zero can be measure-generic (which is directly reflected in the F# implementation). If this were not the case, then any numeric expression $e : \texttt{num}\langle U_1^{x_1} \dots U_n^{x_n}\rangle$ could be tricked to satisfy the type $\texttt{num}\langle\delta'\rangle$ for any δ' just by multiplying e to the polymorphic one and applying the nonexistent rule for polymorphic constant using $\delta = U_1^{-x_1} \dots U_n^{-x_n}\delta'$. Instead, if we multiply something by the polymorphic zero, we are left with just the polymorphic zero, which doesn't break dimensional correctness. The polymorphic zero is also useful as identity for the polymorphic primitive $+$.

3.2.3 Type-inference and unification

In order to build a type-inference algorithm, we should rewrite the set of typing rules in a syntax-directed manner: for our purposes, we will not concentrate on how this is realized and we will focus on the *unification* process, which is crucial for defining a type-inference algorithm and differs from the classical type unification (which is syntactic) because it is semantic.

First of all, we are lucky because unification over the theory of Abelian groups is *unitary*, which means that if two terms can be unified then there exists a most general unifier, which is essential for polymorphism.

Equational unification of δ_1 and δ_2 under $\equiv$ means finding the most general substitution S such that $S(\delta_1) \equiv S(\delta_2)$. This problem can be reduced to the unification of $S(\delta_1 \cdot \delta_2^{-1}) \equiv \mathbf{1}$, thus we will concentrate on this. Suppose we are given a measure expression δ in normal form

$$\delta = u_1^{x_1} u_2^{x_2} \dots u_n^{x_n} \cdot U_1^{y_1} U_2^{y_2} \dots U_m^{y_m},$$

then we can rewrite it as

$$\delta = u_1^{x_1} u_2^{x_2} \dots u_n^{x_n} \cdot (U_1^{y_1/g} U_2^{y_2/g} \dots U_m^{y_m/g})^g,$$

where $g = \gcd(y_1, \dots y_m)$. Unification of δ and $\mathbf{1}$ can be further reduced as the problem of finding $z_1 \dots z_n \in \mathbb{Z}$ such that Equation 8 holds.

$$x_1 z_1 + x_2 z_2 + \dots x_n z_n + g = 0. \tag{8}$$

A more direct alternative approach is followed by Algorithm 1, which iteratively builds the substitution by reducing the exponents until only one variable remains, and if it possible unifies it with the rest of the expression, otherwise reports failure.

Algorithm 1 Algorithm DimUnify

Require: δ is in normal form $u_1^{x_1} \dots u_n^{x_n} \cdot U_1^{y_1} \dots U_m^{y_m}$
Require: Exponents are sorted as $0 < |x_1| \leq \dots \leq |x_n|$

function DimUnify(δ)
  if $n = 0$ **and** $m = 0$ **then**
    return I
  else if $n = 0$ **and** $m \neq 0$ **then**
    abort
  else if $n = 1$ **and** $\forall j. x_1 \mid y_j$ **then**
    return $\{ u_1 \mapsto U_1^{y_1/x_1} \cdots U_m^{y_n/x_1} \}$
  else if $n = 1$ **then**
    abort
  else
    $T = \left\{ u_1 \mapsto u_1 \cdot u_2^{-x_2/x_1} \dots u_n^{-x_n/x_1} \cdot U_1^{-y_1/x_1} \dots U_m^{-y_m/x_1} \right\}$
    $S =$ DimUnify($T(\delta)$)
    return $S \circ T$
  end if
end function

Finally, a type-inference algorithm can be defined by just extending Hindley-Milner's algorithm W: whenever it has to unify types $\texttt{num}\langle\delta_1\rangle$ and $\texttt{num}\langle\delta_2\rangle$ it adds an additional constraint for the (equational) unification of δ_1 and δ_2.

As previously discussed, we restricted our analysis on measure expression with integer exponents only; what should we change to support *rationals* exponent? The appropriate equational theory for this setting is a vector space over $\mathbb{Q}$, which allows to solve unification using Gaussian elimination method.

4 F#'s implementation

In order to analyze how the F# compiler realizes what is discussed in Section 3, the reference implementation considered is the GitHub repository *dotnet/fsharp* [2].

4.1 Internal representation

Syntactically, measure annotations are parsed and represented in the *Abstract Syntax Tree* (AST) by the types `SynMeasure` and `SynRationalConst`; Listing 10 shows the exact definition of those types.

```
type SynMeasure =
    | Named of longId: LongIdent
    | Product of measure1: SynMeasure * measure2: SynMeasure
    | Seq of measures: SynMeasure list
    | Divide of measure1: SynMeasure option * measure2: SynMeasure
    | Power of measure: SynMeasure * power: SynRationalConst
    | One of range: range
    | Anon of range: range
    | Var of typar: SynTypar
    | Paren of measure: SynMeasure

type SynRationalConst =
    | Integer of value: int32
    | Rational of numerator: int32 * denominator: int32
    | Negate of rationalConst: SynRationalConst
    | Paren of rationalConst: SynRationalConst
```

Listing 10: F# parser's internal representation of units of measure.

After parsing/desugaring, measure expression are maintained in a terms of type `Measure`, shown in Listing 11; in the typed syntax tree they can appear as terms of `TType`, alongside normal types.

Measure variables, which play an important role in the type-inference process, are treated as normal type parameters (of type `Typar`) with kind `Measure`, as shown in Listing 12.

4.2 Type-inference and unification

The type-inference procedures involving units of measure follow what has been explained in Section 3: during the constraint solving phase, if a constraint requires that two `TType` must be equal (or one must be a subtype of the other) and they are both `TType_measure`, then the `UnifyMeasure` procedure is called. The latter is just a wrapper function as it internally exploits the `UnifyMeasureWithOne`, which follows the exact same reasoning that we already

```
type Measure =
    | Var of typar: Typar
    | Const of tyconRef: TyconRef
    | Prod of measure1: Measure * measure2: Measure
    | Inv of measure: Measure
    | One of range: range
    | RationalPower of measure: Measure * power: Rational

type TType =
    ...
    | TType_measure of measure: Measure
```

Listing 11: F# type-checker's internal representation of units of measure.

```
type TyparKind =
    | Type
    | Measure

type Typar =
    ...
    member x.Kind = x.typar_flags.Kind
    ...
```

Listing 12: F# compiler's treatment of measure variables.

discussed when we reduced unification of δ_1 and δ_2 to the unification of $\delta_1 \cdot \delta_2^{-1}$ and $\mathbf{1}$. Listing 13 shows the relevant portion demonstrating what we have just explained. The function `UnifyMeasureWithOne` works by cases:

- `ms` is (equivalent to) `1`, and thus there is nothing to do;
- `ms` contains no non-rigid measure variables, and so cannot be unified with `1`;
- `ms` has the form `v^e * ms'` for some non-rigid variable `v`, non-zero exponent `e`, and measure expression `ms'`; therefore, the most general unifier is simply obtained by substituting `v` with `ms'^(-1/e)`.

In addition, it internally calls the function `SubstMeasureWarnIfRigid`, which checks if the unification required some measure variable to be equal to `1`: if it's the case, it emits a warning because the code is less general and it is often unintentional.

4.3 Erasure

During the code generation phase, the compiler strips measures to get the underlying .NET type: this is done using the function `stripTyEqnsWrtErasure`,

```
let SolveTypeSubsumesType ... ty1 ty2 =
    ...
    match ty1 ty2 with
    ...
    | TType_measure ms1, TType_measure ms2 ->
            UnifyMeasures ... ms1 ms2
    ...

let SolveTypeEqualsType ... ty1 ty2 =
    ...
    match ty1 ty2 with
    ...
    | TType_measure ms1, TType_measure ms2 ->
            UnifyMeasures ... ms1 ms2
    ...

let UnifyMeasures ... ms1 ms2 =
    UnifyMeasureWithOne ... (Measure.Prod(ms1, Measure.Inv ms2))

let UnifyMeasureWithOne ... ms = ...
```

Listing 13: F# compiler's relevant code for measure's constraint solving.

shown in Listing 14.

```
type Erasure =
    | EraseAll
    | EraseMeasures
    | EraseNone

val stripTyEqnsWrtErasure: Erasure -> TType -> TType
```

Listing 14: F# compiler's relevant code for measure erasure.

References

[1] *F# Publications — fsharp.org — fsharp.org.* https://fsharp.org/teaching/research#units-of-measure. [Accessed 20-01-2026].

[2] *GitHub - dotnet/fsharp: The F# compiler, F# core library, F# language service, and F# tooling integration for Visual Studio — github.com.* https://github.com/dotnet/fsharp. [Accessed 21-01-2026].

[3] Andrew John Kennedy. *Programming languages and dimensions.* Tech. rep. University of Cambridge, Computer Laboratory, 1996.

[4] Don Syme. “The early history of F#”. In: *Proceedings of the ACM on Programming Languages* 4.HOPL (2020), pp. 1–58.